`otshape` Code Review: Structure, Comments, API, Concepts

Snapshot review of package `otshape` at commit `9760035` (2026-10-31). Read-only evaluation; no code was changed. Claims were checked against symbol usage and the existing documentation. Items marked "unverified" are code-reading observations without a reproduction.

Status (updated 2026-11-01)

The sections after this one are the unchanged original snapshot. File names and line counts in them describe the code as it was at `9760035`.

Step	Status
1. Cheap cleanup	**Partly done** — legacy `buffer.go` and `ShapeEventsRequest` removed; remaining items listed below
2. Split files	**Done** — reorganization by responsibility, including the `lookups.go`, `shape_runtime.go`, `shaper.go`, and `doc.go` issues from section 1
3. Expand the package doc	Open
4. Investigate correctness observations	Open
5. API decisions	Open

Step 2: new file layout

The code was moved verbatim: no renames and no signature, logic, or visibility changes. The set of top-level declarations stayed the same (417), and every moved declaration is byte-identical to the original. The ordinary tests pass and the `shaping_knownbugs` results are unchanged.

File	Contents	Origin
`shape_api.go`	Error sentinels, `Shaper`, `NewShaper`, `Shape`, `shapeMappedRun`, engine selection, `compileShapePlanWithFeatures`	Remainder (604 → 241 lines)
`normalize.go`	Decompose/compose normalization, `planHasGposMark`	`shape_api.go`
`glyph_mapping.go`	`mapRunesToRunBuffer*` (rune→glyph cmap mapping)	`shape_api.go`
`output.go`	Sink writers, `materializeGlyphRecord`	`shape_api.go`
`plan.go`	Plan data model and validation	Remainder (1,183 →

		282 lines)
plan_compile.go	Feature discovery, mask allocation, stage scheduling, compile	plan.go
plan_exec.go	planExecutor, buffer ownership, stage execution; ensureRunMasks, applyFeatureRangesToMasks	plan.go; mask preparation from lookups.go
position_policies.go	Post-GPOS mark-advance zeroing, fallback mark positioning	plan.go
lookups.go	Lookup execution only	Remainder
runbuffer.go	Now also runBuffer.applyLayoutEdit, inheritLayoutEdit	Edit helpers from lookups.go
clusters.go	formInputClusters, mergeClusterRange, runSpan, clusterSpans	shape_runtime.go, output.go
shape_runtime.go	planCompiler, streamIngestor, shapeWorkspace	Remainder
params.go	Params, FeatureRange	Renamed from shaper.go
doc.go	Package documentation only	Remainder
helpers.go	NOTDEF, tracer, errShaper, assert	doc.go

File links in the doc/ guides were updated to match.

Step 1 (partial): legacy API removed

- Deleted buffer.go and its always-skipped buffer_test.go: Buffer, ShapedGlyph, NewBuffer, PREFER_COMPOSED/PREFER_DECOMPOSED, normalizerFor, the findRepresentation search, mapGlyphs, the unused feature lists, and the empty applySubstitutionFeature. These were an early whole-string shaper sketch that never reached GSUB/GPOS.
- Deleted the deprecated, unused ShapeEventsRequest.
- SHAPE-013 now points to the removed representation search as prior art (git show 9760035:otshape/buffer.go).
- Exactly these 19 declarations disappeared. The ordinary tests pass, and the only change in the shaping_knownbugs run is that the skipped TestBufferFunctions no longer exists.

Still open in step 1: the unused wrappers, realignSideArrays, lookupsForStage, stageCount, runBuffer.front/end, the unused planPolicy fields, streamIngestor, and the comment fixes.

Still open from section 1 (not file moves, so deferred to the cleanup): streamIngestor adds nothing over shape_streaming.go, and pipeline.go is a vague name for the I/O types. The runbuffer.go repetition remains; inheritLayoutEdit[T] now sits in the same file as the four typed applyEdit* copies.

Summary

The pipeline design is reasonable: fonts and engines are passed in explicitly, input and output stream through interfaces, and the internal side arrays stay aligned when glyphs are edited. The newest comments, which state contracts, are good. The main weaknesses:

- **The public API exposes internals.** Positions, masks, and flush modes show internal state, and several exported items are dead.
- **Two large files mix unrelated jobs.** `shape_api.go` and `plan.go` need splitting.
- **The package doc says almost nothing.** The concept docs in `doc/*.md` are honest and detailed, but godoc readers never see them.

1. Functionality and locality per file

File	Assessment
`mark_attachments.go`, `input_events.go`, `plan_stack.go`, `shape_streaming.go`	**Good.** Each has one job, with clear invariants.
`engine_api.go` / `engine_internal.go`	**Good split** between the public hook interfaces and the private adapters behind them.
`shape_api.go` (604 lines)	**Weakest.** Mixes the `Shaper` type and errors, the `Shape` loop, hook calling order, engine selection, plan-compile glue, rune→glyph mapping, normalization (decompose/compose), and output writing. Normalization and output should be their own files.
`plan.go` (1,183 lines)	Holds both plan **compilation** (reading the font's features, masks, stages) and plan **execution** (running GSUB/GPOS, mark-advance zeroing, fallback mark positioning). Should be split into three files.
`lookups.go`	Lookup execution, but also mask setup, feature-range masks, and `runBuffer.applyLayoutEdit`, which is a buffer method.
`shape_runtime.go`	Grab-bag. `streamIngestor` only forwards to `shape_streaming.go` and adds nothing. Cluster logic (`formInputClusters`, `mergeClusterRange`) would fit better in a `clusters.go`.
`shaper.go`	Misleading name: holds `Params` and `FeatureRange`, while the `Shaper` type lives in `shape_api.go`. `doc.go` also holds helpers (`assert`, `tracer`, `errShaper`).
`runbuffer.go`	Cohesive but very repetitive: 10 side arrays × Ensure/Use/Reserve/Append/Edit, plus four typed `applyEdit*` copies, while `lookups.go` already has a generic `inheritLayoutEdit[T]`.
`buffer.go`	**Legacy; dead in production, used only by tests.** Still exports `Buffer`, `ShapedGlyph`, `NewBuffer`, and `PREFER_COMPOSED`/`PREFER_DECOMPOSED`, which show up in godoc next to the real API. Its font-aware representation logic is what SHAPE-013 lacks today.
`language.go`	About half of the `script2opentype` keys are Unicode script names

	("Thai", "Malayalam") rather than ISO 15924 codes, so they never match (already recorded in TESTING.md). Only 11 languages are mapped.

Dead or never-activated code

- `runBuffer.front` / `runBuffer.end`.
- `realignSideArrays`, `lookupsForStage`, `stageCount`, and the `writeRunBuffer*`, `normalizeRuneStream`, `composeRuneStream`, and `mapRunesToRunBuffer` wrappers: used only by tests, or not at all.
- `planPolicy.Strict`, `ZeroMarks`, `FallbackMarkPos`, `VarIndex`, and `Notes` are never set or read in production, so their branches cannot run.
- `UnsafeFlags` is never populated, so `isBreakSafeCut` always returns true (already documented in STREAMING_API.md).

Correctness observations not yet in SHAPING_FINDINGS.md (unverified)

Both need a reproduction and an ID before becoming findings.

1. **Lookup order across features.** `compileTableProgram` gives every default feature its own stage (`stageNo++` per tag), and user-enabled features get one stage each, sorted by tag. Lookups therefore run feature by feature (locl, ccmp, rlig, …; for GPOS mark, mkmk, …, kern). The OpenType spec and HarfBuzz instead run all features of one stage in LookupList index order. Fonts whose `calt`/`liga` or `kern`/`mark` lookups depend on each other could shape differently.
2. **Engine features overwrite user settings.** `planFeaturePlanner.AddFeature` and `DisableFeature` replace `togglesByTag[tag]` entirely. If a user disables a feature that the engine enables in `CollectFeatures` (e.g. `calt` or `liga` in `otarabic`), the feature stays active. Masks still respect user ranges; the on/off decision does not.

2. Code comments

Strengths

Recent comments explain *why*, not just *what*. Good examples: `FeatureRange`, `GlyphSink`, the error semantics of `Shape`, `formInputClusters`, `mergeClusterRange`, `applyFeatureRangesToMasks`, the random-seed reasoning in `applyLookupSpanInternal`, and `mark_attachments.go`.

Gaps

- **The most intricate code has almost no comments:** `compile` / `compileTableProgram` (stage meaning, required-feature-at-slot-0 convention, user toggles vs. defaults), mask encoding, and the `-target-2` index encoding in `applyLookupIsolatedSpan. annotate.go` has no comments at all.
- **Many exported items lack doc comments:** `LayoutGSUB`/`LayoutGPOS`, all `Normalization*`, `Feature*` flag and `ShaperConfidence*` constants, the fields of `SelectionContext` and `ResolvedFeature`, and every method of `FeaturePlanner` and

`RunContext`. For `RunContext`, nothing states that out-of-range calls silently do nothing, how `InsertGlyphs` inherits clusters, or whether `Codepoint` remains meaningful after GSUB.

Stale or wrong comments

- The package doc links `[Shape]` and `[ShapeEvents]`, but neither exists as a package function, so godoc renders plain text. Should be `[Shaper.Shape]` and `[Shaper.ShapeEvents]`.
- `GlyphRecord` says combining-mark grouping "is currently incomplete; see SHAPE-011", but SHAPE-011 is resolved.
- `ShapingEnginePreprocessHook` is documented as "pre-normalization", but `shapeMappedRun` calls it after normalization and mapping.
- `LowWatermark` is described as a "minimum cursor position"; there is no cursor. It is the minimum number of raw runes to flush.
- `FlushExplicit` mentions signalling "from RuneSource".
- `ShapeEvents` says "output order" where `Shape` says "logical shaping order".
- `NewShaper` refers to "Nil entries in shapers"; the parameter is `engines`.
- `FeatureGlobal` and `FeatureGlobalSearch` are accepted but ignored, and nothing says so.
- `AddGSUBPause` reads as if it inserts the pause at the call site, but pauses are appended after all lookups. SHAPER_INTERFACES.md says so; godoc does not.
- Typos: "Unicde", "wether", "Devangari", "Hanguli".
- Mixed error prefixes: `otshape:` vs. `OpenType text shaping:`.

3. Top-level API

Sound

No global registry; engines are injected explicitly. Input is a streaming `RuneSource` / `InputEventSource`, output a `GlyphSink`. Output is in logical order, clusters are scalar indexes. Errors are exported sentinel values, and partial-output behavior is documented.

Problems, roughly by importance

1. **`GlyphRecord` exposes internals.**
2. `Pos` is an `otlayout.PosItem`, carrying `AttachTo` (an index into the current internal run, meaningless after a partial flush), `AttachKind`, `AnchorRef`, and `MarkAttachmentPending`.
3. `Mask` uses a plan-private bit layout.
4. `UnsafeFlags` is always zero, and the constants needed to read it are not exported.

A small public position struct (advances and offsets) would make the output a stable contract. 2. **`FlushBoundary` makes no visible difference to the sink.** Both modes call `WriteGlyph` once per glyph in the same order. `FlushExplicit` is exported but rejected. 3. **Dead or deprecated exports:** `ShapeEventsRequest` (deprecated, unused), plus legacy `Buffer`, `ShapedGlyph`, `NewBuffer`, and `PREFER_*`. The repository has no tags, so removing them now is cheap. 4.

`Shape` and `ShapeEvents` duplicate their loop, and feature scoping differs between them (ranges vs. push/pop; ranges are forbidden in events mode). `ShapeEvents` compiles a new plan on every push without caching and never frees plans (IDs are `uint16`). 5. **No plan reuse.** Every call recompiles the plan, including joiner classes; nothing caches by (font, script, language, direction, features). This gets expensive for many short segments. 6. **No explicit OpenType script/language tag override** (e.g. `deva` vs. `dev2`, or languages outside the 11 mapped); callers depend on the partly broken maps. 7. **The engine interface is large and partly ignored:** - 11 optional hook interfaces, several used only by Arabic. - `FeatureFlags` bits are ignored. - Feature values use three types: `uint32` in `AddFeature`, `int` in `Arg`, `int` in `Value`. - Both a package-level `New()` and `Engine.New()` exist. - `RunContext` writes fail silently, which hides engine bugs. 8. **Unstated properties:** whether `Shape` is safe for concurrent use. Random alternates are seeded with `rand.Uint64()` per call, so identical calls can give different output, unlike HarfBuzz. 9. **`BufferOptions`:** a zero value means "use the default", so 0 cannot be requested explicitly. The agreed buffer contract will reshape this anyway.

4. Explanation of the overall concepts

The `doc/` files are thorough and careful to separate current behavior from agreed intent. SHAPING_OVERVIEW, SHAPER_INTERFACES, and STREAMING_API cover the actual hook order, data flow, cut selection, and what is guaranteed vs. merely observed.

The package doc in `doc.go` is about 15 lines with broken links. It does not explain:

- what a plan is;
- how engines are selected;
- the hook order;
- normalization;
- cluster semantics (only described on `GlyphRecord`);
- position units, or that `XAdvance` already includes the nominal advance;
- streaming limits (SHAPE-015);
- known limitations.

It does not link to the `doc/` files, and there are no `Example` functions. An engine author must read the markdown or the source to learn the hook order. The engine package docs (`otcore`, `othebrew`) are brief but accurate.

Suggested next steps

1. **Cheap cleanup:** delete or internalize `buffer.go`, `ShapeEventsRequest`, the dead wrappers, unused policy fields, and `front/end`. Fix stale and broken comments; document exported constants and interface methods.
2. **Split files:** `normalize.go` and `output.go` out of `shape_api.go`; plan compile vs. execute out of `plan.go`.
3. **Expand the package doc:** short concept overview, links to `doc/*.md`, one runnable `Example`.
4. **Investigate and record** the two new correctness observations (stage / lookup order;

engine overriding user features) as findings with reproductions.

5. **API decisions for the maintainer:** a public position type instead of `otlayout.PosItem`; whether to keep `Mask`, `UnsafeFlags`, and `FlushBoundary` public; plan caching; explicit tag overrides.